UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERIA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS

ALGORITMOS DE REEMPLAZO

Asignatura: **Sistemas Operativos**

Docente: **Jose Mauro Pillco Quispe**

Alumno:**Marco Antonio Arapa Salazar**

Código: **225417**

Cusco - 2025

INTRODUCCIÓN

Los sistemas operativos modernos son responsables de administrar de manera eficiente los recursos del hardware, tales como la memoria principal, el procesador, los dispositivos de entrada y salida y los mecanismos de interrupción. Uno de los desafíos más importantes en la administración de memoria es la gestión de memoria virtual, particularmente el proceso de reemplazo de páginas cuando la memoria física disponible no es suficiente para alojar todas las páginas activas de los procesos en ejecución.

En este contexto, los algoritmos de reemplazo de páginas constituyen una pieza fundamental en el diseño de los sistemas operativos. Estos algoritmos determinan qué página debe ser removida de la memoria principal cuando ocurre un fallo de página (page fault), es decir, cuando un proceso intenta acceder a una página que no se encuentra actualmente cargada en memoria.

El presente trabajo consiste en el desarrollo e implementación en lenguaje C++ de múltiples algoritmos clásicos de reemplazo de páginas, incluyendo FIFO, Óptimo, LRU, LFU, NFU, CLOCK, Segunda Oportunidad y NRU. Además, el programa no solo simula el comportamiento de dichos algoritmos, sino que también integra la medición de métricas reales del sistema operativo Windows, tales como consumo de memoria, tiempo de ejecución, interrupciones de hardware y llamadas al sistema.

Este enfoque permite no solo comparar el desempeño lógico de cada algoritmo en términos de fallos y aciertos (hits), sino también analizar su impacto en recursos reales del sistema, proporcionando una visión más completa del comportamiento de los algoritmos desde una perspectiva tanto teórica como práctica.

ANTECEDENTES

La memoria virtual fue introducida como solución al problema de la limitada capacidad de la memoria física. Permite que los procesos crean disponer de un espacio de direcciones mayor que la memoria RAM disponible, delegando al sistema operativo la responsabilidad de cargar y descargar páginas entre memoria principal y almacenamiento secundario.

Cuando un proceso referencia una página que no está en memoria, se produce un fallo de página. En ese momento, el sistema operativo debe decidir cuál de las páginas actualmente cargadas será reemplazada. Para ello, se han desarrollado diversos algoritmos a lo largo de la historia de los sistemas operativos.

Entre los algoritmos más conocidos se encuentran:

- FIFO (First In, First Out)
- Óptimo (Belady)
- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- NFU (Not Frequently Used)
- CLOCK
- Segunda Oportunidad
- NRU (Not Recently Used)

Cada uno de estos algoritmos surge como respuesta a limitaciones observadas en algoritmos anteriores. Por ejemplo:

- FIFO es simple pero puede sufrir la anomalía de Belady.
- El algoritmo Óptimo minimiza fallos pero es imposible de implementar en sistemas reales porque requiere conocimiento

futuro.

- LRU intenta aproximarse al óptimo utilizando información histórica.
- CLOCK y Segunda Oportunidad reducen el costo computacional de LRU usando bits de referencia.
- NRU clasifica páginas según su uso reciente.

En sistemas reales, la implementación de estos algoritmos busca un equilibrio entre eficiencia computacional, consumo de memoria y reducción de fallos de página.

El desarrollo del presente programa permite estudiar estos algoritmos de manera comparativa bajo condiciones controladas, generando secuencias aleatorias de referencias y observando su comportamiento bajo diferentes cantidades de marcos de memoria.

DESCRIPCIÓN DEL PROGRAMA

El programa fue desarrollado en lenguaje C++ utilizando librerías estándar y librerías específicas del sistema operativo Windows, tales como:

- `<windows.h>`
- `<psapi.h>`
- `<chrono>`
- `<vector>`
- `<algorithm>`
- `<random>`

La estructura general del programa se divide en los siguientes componentes:

3.1 Generación de la cadena de referencias

Se genera una secuencia aleatoria de páginas utilizando un generador de números pseudoaleatorios. Las páginas se encuentran en un rango de 0 a 9 y la cantidad de referencias es ingresada por el usuario.

3.2 Estructuras de datos

Se definen dos estructuras principales:

Resultado

- Número de fallos
- Número de hits

- Tabla de estados de marcos
- Estado de cada referencia (F o H)

Metricas

- Memoria utilizada (KB)
- Tiempo total de ejecución (ms)
- Número de syscalls
- Número de interrupciones de hardware
- Tiempo en modo kernel

3.3 Implementación de algoritmos

Se implementan ocho algoritmos de reemplazo de páginas:

1. FIFO
2. Óptimo
3. LRU
4. LFU
5. NFU
6. CLOCK
7. Segunda Oportunidad
8. NRU

Cada algoritmo retorna un objeto de tipo `Resultado` que permite comparar desempeño.

3.4 Medición de métricas reales

El programa utiliza funciones del sistema Windows como:

- `GetProcessMemoryInfo`
- `GetProcessIoCounters`
- `GetProcessTimes`
- `NtQuerySystemInformation`

Esto permite obtener:

- Uso real de memoria del proceso
- Llamadas de entrada/salida
- Interrupciones del sistema
- Tiempo de ejecución en modo kernel

Esta característica distingue el programa de una simple simulación académica, convirtiéndo en una herramienta híbrida entre simulador y monitor de rendimiento.

¿CÓMO FUNCIONA EL PROGRAMA?

El funcionamiento del programa se puede describir en las siguientes etapas:

Paso 1: Entrada de datos

El usuario ingresa:

- Número de referencias de página
- Número de marcos disponibles
- Algoritmo a ejecutar (o todos)

Paso 2: Generación de referencias

Se crea un vector de enteros representando la secuencia de acceso a memoria.

Paso 3: Ejecución del algoritmo

Para cada referencia:

- Se verifica si la página está en memoria.
- Si está → se registra un hit.
- Si no está → se produce un fallo y se aplica la política de reemplazo correspondiente.
- Se actualizan los marcos.
- Se almacena el estado en la tabla.

Paso 4: Medición de métricas

Antes y después de ejecutar el algoritmo:

- Se registran interrupciones del sistema.
- Se mide el tiempo de ejecución.
- Se obtiene el consumo de memoria.
- Se contabilizan las operaciones de I/O.

Paso 5: Presentación de resultados

El programa imprime:

- Número total de fallos
- Número total de hits
- Métricas del sistema

Si el usuario selecciona la opción 9, se ejecutan todos los algoritmos consecutivamente para comparación.

¿PARA QUÉ SIRVE ESTE PROGRAMA?

Este programa tiene múltiples aplicaciones académicas y técnicas:

5.1 Propósito académico

- Permite comprender el funcionamiento interno de los algoritmos de reemplazo.
- Facilita comparar desempeño bajo las mismas condiciones.
- Permite observar diferencias entre algoritmos teóricos y prácticos.

5.2 Análisis comparativo

Permite estudiar:

- Qué algoritmo produce menos fallos.
- Cómo influye el número de marcos.
- Diferencias entre algoritmos predictivos y heurísticos.

5.3 Medición de impacto real

Al integrar métricas reales del sistema:

- Se puede analizar si algoritmos más complejos consumen más tiempo de CPU.
- Se puede observar el impacto en interrupciones y modo kernel.
- Se relaciona teoría con comportamiento real del sistema operativo.

5.4 Desarrollo profesional

Este tipo de implementación fortalece conocimientos en:

- Gestión de memoria
- Programación en C++
- Interacción con API del sistema operativo
- Análisis de rendimiento

COMO LO IMPLEMENTE

La implementación se realizó siguiendo una arquitectura modular y estructurada.

1. Generación de la cadena de páginas

Se utiliza un generador pseudoaleatorio basado en `mt19937`, que genera valores enteros entre 0 y 9. Esto simula accesos a diferentes páginas de memoria.

La función `generarCadena(int n)` crea un vector de tamaño n que representa la secuencia de referencias.

---

2. Estructuras principales

Se definieron dos estructuras fundamentales:

Estructura Resultado

Contiene:

- Número total de fallos.
- Número total de hits.
- Tabla de estados de los marcos.
- Vector que indica si cada referencia fue fallo o hit.

Esta estructura permite almacenar toda la información necesaria para análisis posterior.

Estructura Metricas

Contiene:

- Memoria usada (KB).
- Tiempo total de ejecución.
- Número de syscalls.
- Número de interrupciones.
- Tiempo en modo kernel.

---

3. Implementación de algoritmos

Cada algoritmo fue implementado como una función independiente que recibe:

- Vector de páginas.
- Número de marcos disponibles.

Los algoritmos implementados son:

- FIFO
- Óptimo
- LRU
- LFU
- NFU
- CLOCK
- Segunda Oportunidad
- NRU

Cada uno sigue su lógica específica de reemplazo:

- FIFO reemplaza la página más antigua.
- Óptimo reemplaza la página cuyo uso futuro esté más lejano.
- LRU reemplaza la menos recientemente usada.
- LFU reemplaza la menos frecuentemente usada.
- CLOCK usa bits de referencia simulando comportamiento circular.
- NRU clasifica según uso reciente.

---

4. Captura de métricas del sistema

Antes de ejecutar el algoritmo se obtiene:

- Número de interrupciones del sistema.

Después de la ejecución se vuelve a capturar:

- Interrupciones.
- Uso de memoria.
- Operaciones de I/O.
- Tiempo en modo kernel.

La diferencia entre valores antes y después permite medir impacto real.

RESULTADOS OBTENIDOS

Al ejecutar el programa, se obtienen resultados en dos niveles:

Nivel 1: Resultados del algoritmo

- Número total de fallos.
- Número total de hits.

Esto permite evaluar la eficiencia del algoritmo en términos de reemplazo de páginas.

Nivel 2: Métricas del sistema

- Memoria utilizada por el proceso.
- Tiempo de ejecución en milisegundos.
- Número de llamadas al sistema.
- Interrupciones de hardware registradas.
- Tiempo de CPU en modo kernel.

Estos resultados permiten observar no solo cuál algoritmo produce menos fallos, sino cuál tiene mayor impacto en recursos reales.

¿QUÉ PASA CUANDO SE EJECUTA EL PROGRAMA?

Cuando el usuario ejecuta el programa:

1. Ingresa la cantidad de referencias.
2. Ingresa el número de marcos.
3. Selecciona el algoritmo.

El sistema genera automáticamente la cadena de páginas y procede a simular el comportamiento del algoritmo seleccionado.

Durante la ejecución:

- Se evalúa cada referencia.
- Se determina si hay hit o fallo.
- Se actualizan los marcos.
- Se registran métricas.

Finalmente, se muestran resultados en consola con estadísticas completas.

Si se selecciona la opción "TODOS", el programa ejecuta secuencialmente todos los algoritmos bajo las mismas condiciones, permitiendo comparación directa.

ANALISIS DE LOS RESULTADOS

El análisis permite observar diferencias claras entre algoritmos.

1. El algoritmo Óptimo suele producir el menor número de fallos, ya que utiliza conocimiento futuro.

2. LRU tiende a aproximarse al comportamiento del Óptimo.

3. FIFO puede presentar más fallos debido a su simplicidad.

4. CLOCK y Segunda Oportunidad ofrecen un balance entre eficiencia y bajo costo computacional.

5. LFU puede comportarse bien en patrones repetitivos.

6. NRU simplifica decisiones usando bits de referencia.

En términos de métricas reales:

- El tiempo de ejecución suele variar ligeramente entre algoritmos.

- Algoritmos con mayor lógica (Óptimo, LRU) pueden consumir más tiempo de CPU.

- El consumo de memoria del proceso se mantiene relativamente estable.

- Las interrupciones del sistema no dependen directamente del algoritmo, sino del entorno del sistema.

Esto demuestra que la complejidad algorítmica impacta principalmente el tiempo de ejecución más que otros recursos del sistema.

CONCLUSIONES

El desarrollo e implementación de múltiples algoritmos de reemplazo de páginas permitió comprender profundamente el funcionamiento interno de la gestión de memoria en sistemas operativos.

Se concluye que:

1. No existe un algoritmo perfecto aplicable a todos los casos.
2. El algoritmo Óptimo es teóricamente el mejor, pero impracticable en sistemas reales.
3. LRU ofrece un buen equilibrio entre eficiencia y viabilidad.
4. Algoritmos como CLOCK reducen costos computacionales.
5. La medición de métricas reales permite vincular teoría con práctica.

El programa demuestra que la eficiencia de un algoritmo no solo debe medirse en número de fallos, sino también en impacto computacional real.

Además, el proyecto fortalece conocimientos en:

- Programación en C++.
- Gestión de memoria virtual.
- Interacción con APIs del sistema operativo.
- Análisis de rendimiento.

En conclusión, el trabajo cumple con el objetivo de simular, comparar y analizar algoritmos de reemplazo de páginas desde una perspectiva integral, combinando teoría, implementación práctica y evaluación del comportamiento real del sistema.